

Sunny Tea House 任务交付文档

一、交付内容

交付物	地址/位置
在线演示 (Demo)	https://tyza66.github.io/sunny-tea-house/
项目仓库	https://github.com/tyza66/sunny-tea-house
项目文档	本目录 docs/*.md 与仓库 README.md , PDF 版见 docs/交付文档.pdf

在线演示由 GitHub Pages 静态托管。页面在没有 `/api` 时自动使用本地示例文案，保证“选择感受 → 选择平台 → 生成 → 编辑 → 复制并跳转”闭环在手机浏览器可直接跑通；真实 AI 模式仍走服务端 DeepSeek。

二、Demo 交互闭环

- 感受选择**：6 个感受标签，最多选 2 个，前端与后端双重校验。
- 平台选择**：Google 与小红书两个平台，另有独立于界面的评价语言选择。
- AI 智能生成**：Google 输出自然的北美英语评论；小红书输出中文“种草笔记”风格，带 Emoji 与分段。真实模式调用 DeepSeek，演示模式使用本地模板并明确标注。
- 编辑与流转**：textarea 可二次编辑；勾选“确认真实体验”后一键复制，并同步打开对应平台入口。复制失败时选中文本并提供手动平台链接。

二·附、界面预览

界面为「青瓷」瑞士网格极简风格：冷骨白底配竖向基准引导线，全站只有一个青瓷釉绿信号色；标题用无衬线细字重，序号与字数用等宽字体，右上角可切换深色主题。



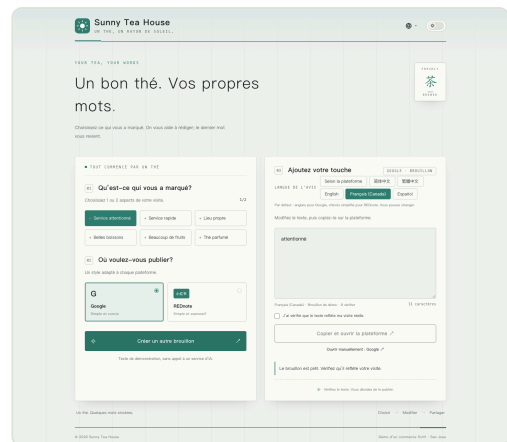
手机 · 简体中文



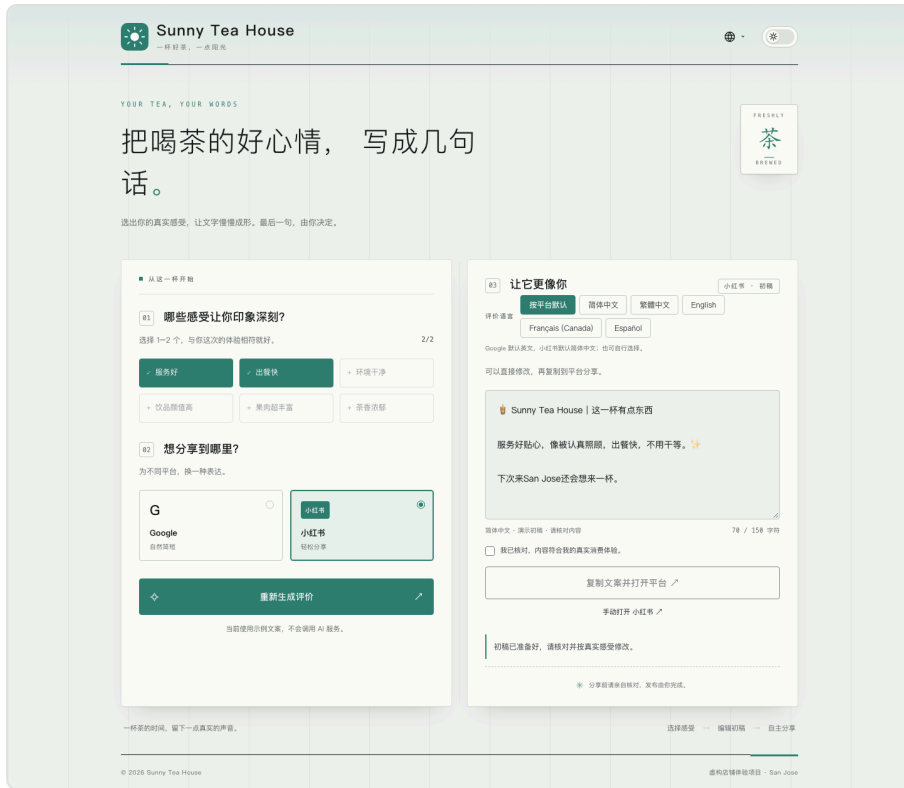
桌面 · 简体中文



手机 · 法语



桌面 · 法语



生成结果（小红书风格）

三、AI 辅助编程工具使用策略与提效心得

本任务使用 Codex 作为主要编码工具，策略如下：

- 先让 AI 读取目标仓库、README、配置与测试，再开始改动，避免凭印象重写。
- 每次改动保持小而可验证：核心功能、测试、文档、部署分步提交，重要节点 push。
- 用自动化测试守住行为边界：后端单元/HTTP 测试、语言测试、Playwright 手机流程测试。
- 明确给 AI 限定范围：只改与任务相关的模块，不顺手重构无关代码。
- 对 AI 生成内容做人工复核：检查密钥是否进前端、跳转是否真实、文案是否可能虚构消费事实。

提效心得：把“可验证”写在前面比让 AI 直接写完整方案更有效；提示词工程本身也可交给 AI 起草，但平台口吻、字数与事实边界必须由人明确。

四、Prompt 设计思路与调优过程

Prompt 在 `server/reviews.js` 中按“平台声音 + 感受落点 + 语言 + 事实边界”分层组装：

角色：你是一位住在北美的真实顾客，正给常去的茶饮店写一条客观、自然的 Google 评价。
结构：用 3-4 个完整的句子写成一段，第一人称。语气自然克制、直白可信。
禁用：标题、Markdown、项目符号、Emoji、感叹号、广告词。

小红书则单独设置“种草笔记”声音：

角色：你是一位在小红书分享好物的普通用户，正在写一条真实、轻松的种草笔记。
长度：全文不超过 150 个 Unicode 字符（含标点、空格与 Emoji）。

Emoji: 按所选感受适当点缀, 例如出餐快⚡、茶香浓郁🍵。

关键调优点:

- **语言与平台解耦**: Google 默认英文、小红书默认中文, 但都允许单独切换五种语言, 避免“平台=语言”的耦合。
- **只写所选感受**: 提示词明确禁止提到未选择的优点或具体细节, 防止模型把示范里的内容带进结果。
- **防虚构**: 禁止编造饮品名称、价格、排队时间、购买次数; 最终仍由顾客勾选确认后自行发布。
- **长度兜底**: 小红书生成后代码检查 150 字, 超限只修正一次, 避免无限重试产生费用。
- **输出纯净**: 只输出初稿本身, 禁止解释或翻译; 结果以纯文本渲染, 不当作 HTML 执行。

五、附加题: 企业微信群机器人推送

服务端在评价初稿成功返回后, 异步完成一条轻量自动化工作流:

1. 调用 DeepSeek 生成两部分内容: 该评论的**中文摘要**与**给商家的回复草稿**。
2. 将初稿、平台、感受与上述内容拼装为企微文本消息, POST 到群机器人 Webhook。
3. 消息标注“评价初稿 · 尚未发布”, 避免误导商家。

关键实现位于 `server/reviews.js` 的 `notifyWechat`:

```
const message = `【评价初稿 · 尚未发布】\n${config.store.name} · ${input.platform}\n感受:\n${input.tags.join('、')} \n\n${content}\n\n${followup}`;\n// 企业微信文本上限 2048 字节, 按 Unicode 字符累计截断, 避免切坏汉字。\nlet safeText = '';\nfor (const char of message) { if (Buffer.byteLength(safeText + char, 'utf8') > 2000) break;\nsafeText += char; }
```

该流程默认关闭, 仅在 `ENABLE_WECHAT_NOTIFY=true` 且配置有效 Webhook 时启用; 发送失败只记录日志, 不影响顾客端已生成的初稿。

说明: 本次交付未接入真实企业微信群机器人, 因此没有群消息截图; 数据拼装与调用逻辑已按上面代码实现, 并有测试覆盖 (`tests/api.test.js` 中“企业微信默认不发送, 启用时生成摘要并发送标记为初稿的消息”)。

六、API Key 安全与用量成本处理

- 密钥只存在于服务端 `.env` 或 Netlify 环境变量, 从不进入 Vue 代码、浏览器请求或版本库; 不使用 `VITE_` 前缀。
- 浏览器只调用本站 `/api/reviews`, DeepSeek 的 `Authorization` 头由服务端组装。
- 接口固定官方 `https://api.deepseek.com/chat/completions`, 不接受前端指定任意地址。
- 全站 CSP 只允许本站资源; 单 IP 每分钟限 10 次生成; 请求体限制 8KB; 单次 AI 调用 35 秒超时。
- 用量控制: 短评价场景关闭推理模式、限制 `max_tokens`、小红书超长只修正一次、通知流程仅在商家启用时调用。

七、部署与验证

- 本地演示：`npm run dev` 或 `npm run build && npm start`。
- Netlify 真实 AI：`netlify.toml` + `netlify/functions`，后台配置 `DEEPSEEK_API_KEY`。
- GitHub Pages 静态演示：`npm run build` 后推送，由 `.github/workflows/pages.yml` 自动发布。
- 验证：`npm test` (28 项，含静态演示回退 7 项)、`npm run test:e2e` (13 项)、`npm run build` 均通过；在线链接已在手机视口下做生成、编辑、复制与跳转检查。
- GitHub Pages 只发布静态产物，Actions 注入密钥等于公开泄露，页面也无法同步等待 workflow，因此真实 AI 只能放在 Netlify Functions 或自有服务器。实测免密钥公共文本接口超时与内容错配严重，未接入，静态演示沿用本地模板文案。

八、实际耗时

按本机文件时间戳与 Git 提交记录统计：首个源码文件生成于 16:29，完整复刻与测试首次提交于 16:44（约 15 分钟）；静态在线演示、回退修复、部署验证与文档于 17:18 完成（累计约 50 分钟）；本 PDF 整理与线上复核约 10 分钟；「墨与柿」前端换肤、层叠与截图缺陷修复、文档与截图重刷于 18:32 完成；「青瓷」二次换肤、测试与文档同步于 18:50 完成（再增约 18 分钟）。全程约 2 小时 20 分钟，均为 AI 辅助下的实际执行时间，不含需求阅读、人工复盘与真实账号联调等待时间。